

UNIVERSIDAD TÉCNICA DE AMBATO

Facultad de Ingeniería en Sistemas, Electrónica e Industrial

Carrera de Ingeniería de Software



INFORME ACADÉMICO

Recorridos de Arboles Binarios

Asignatura: Estructura de Datos

Docente: Ing. Caiza Caizaguano Jose Ruben

Estudiante: Tituaña Calapiña Diana Pamela

Nivel/Paralelo: Tercero - "B"

Fecha: 08/05/2026

1. Introducción

Los arboles binarios constituyen una de las estructuras de datos más importantes dentro de la programación orientada a algoritmos. Su capacidad para organizar información de forma jerárquica los convierte en herramientas imprescindibles en áreas como la construcción de compiladores, los motores de búsqueda, los sistemas de archivos y los procesamiento de expresiones matemáticas.

El presente informe documenta la implementación y análisis de los principales recorridos sobre arboles binarios: Preorden, Inorden, Postorden y BFS. Se utiliza dos lenguajes de programación C++ y Java, permitiendo observar las similitudes y diferencias entre ambas implementaciones. Además, se desarrollan operaciones de conteo y se aplica el concepto a un caso práctico.

2. Objetivos

Objetivo General

Implementar y analizar los principales recorridos de arboles binarios mediante la programación en C++ y Java, aplicando los conceptos de recursividad y estructuras de datos para resolver problemas prácticos relacionados con la organización jerárquica de información en un sistema.

Objetivos específicos

- Desarrollar funciones de recorrido (Preorden, Inorden, Postorden y BFS) utilizando recursividad y cola como estructuras de control.
- Añadir funciones auxiliares de conteo de nodos totales y hojas de árbol.
- Aplicar los recorridos de arboles binarios a un caso real, modelando los módulos de un sistema web como un árbol binario.

3. Marco Teórico

3.1. Tipos de recorridos

Tabla I. Tipo de recorridos de árboles binarios

Recorrido	Método de Implementación	Orden	Uso típico
Preorden	Recursivo	raíz-izquierda-derecha	Obtener datos en orden ascendente
Inorden	Recursivo	izquierda-raíz-derecha	Crear una copia del árbol o mostrar estructura jerárquica
Postorden	Recursivo	izquierda-derecha-raíz	Liberar memoria del árbol o procesar hijos antes que el padre
BFS	Iterativo con cola	Nivel a nivel con cola	Mostrar el árbol por niveles o búsqueda de camino más corto

3.2. Conceptos clave

- **Nodo:** Cada elemento del árbol que contiene un dato y punteros a sus hijos.
- **Raíz:** El nodo superior del árbol, no tiene padre.
- **Hoja:** Nodo que no tiene hijos (sus punteros izquierdo y derecho son null).
- **Altura:** Número de niveles del árbol (desde la raíz hasta la hoja más profunda).

4. Documentación

4.1. Árbol Original y Recorridos

El siguiente árbol binario está constituido por la raíz con el nodo 10 y este está distribuido en 3 niveles que están compuestos por 7 nodos.

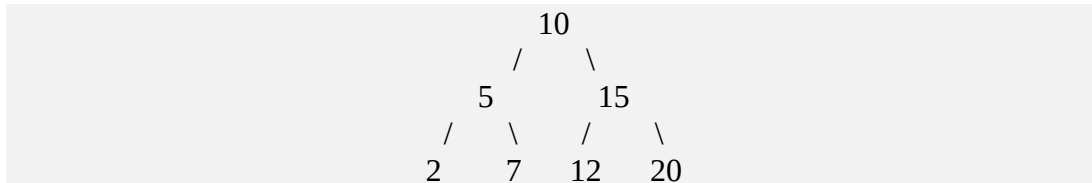


Tabla II. Recorrido Árbol Original

Recorrido	Resultado	Orden
Preorden	10 5 2 7 15 12 20	raíz-izquierda-derecha
Inorden	2 5 7 10 12 15 20	izquierda-raíz-derecha
Postorden	2 7 5 12 20 15 10	izquierda-derecha-raíz
BFS	10 5 15 2 7 12 20	Nivel a nivel con cola

4.1.1. Implementación en C++ y Java

Las funciones de recorrido son idénticas en lógica para ambos lenguajes. Las diferencias son sintácticas: punteros en C++ frente a referencias en Java. Nullptr frente a null y cout frente a System.out.print.

C++

```
// ===== RECORRIDOS =====  
void preorden(Nodo* raiz) {  
    if (raiz == nullptr) return;  
    cout << raiz->dato << " ";  
    preorden(raiz->izquierda);  
    preorden(raiz->derecha);  
}
```

Java

```
// ===== RECORRIDOS =====  
public static void preorden(Nodo raiz) {  
    if (raiz == null) return;  
    System.out.print(raiz.dato + " ");  
    preorden(raiz.izquierda);  
    preorden(raiz.derecha);  
}
```

El recorrido BFS utiliza una cola para garantizar el procesamiento nivel a nivel, siendo el único recorrido no recursivo de los cuatro implementados.

C++

```

void bfs(Nodo* raiz) {
    if (raiz == nullptr) return;

    queue<Nodo*> cola;
    cola.push(raiz);

    while (!cola.empty()) {
        Nodo* actual = cola.front();
        cola.pop();

        cout << actual->dato << " ";

        if (actual->izquierda != nullptr) cola.push(actual->izquierda);
        if (actual->derecha != nullptr) cola.push(actual->derecha);
    }
}

```

Java

```

public static void bfs(Nodo raiz) {
    if (raiz == null) return;

    Queue<Nodo> cola = new LinkedList<>();
    cola.add(raiz);

    while (!cola.isEmpty()) {
        Nodo actual = cola.poll();
        System.out.print(actual.dato + " ");

        if (actual.izquierda != null) cola.add(actual.izquierda);
        if (actual.derecha != null) cola.add(actual.derecha);
    }
}

```

4.2. Árbol con nodos adicionales

Se agregaron 6 nodos nuevos en el árbol; 1 y 3 como hijos del nodo 2, 18 y 25 como hijos del nodo 7, y 30 y 42 hijos de nodo 12. Al final el árbol tiene un resultado de 13 nodos en 4 niveles.

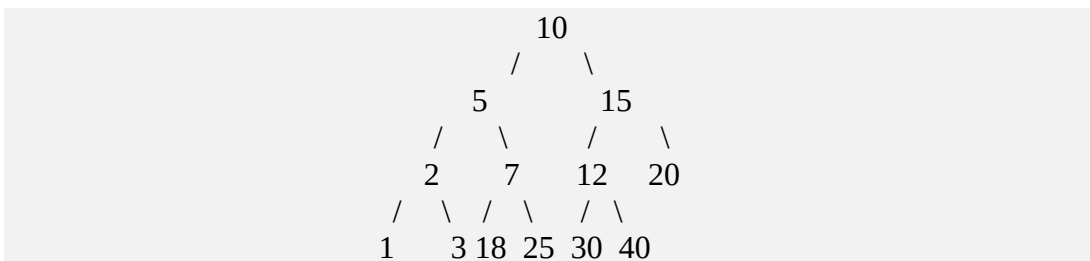


Tabla III. Recorrido Árbol Modificado

Recorrido	Resultado
Preorden	10 5 2 1 3 7 18 30 42 25 15 12 20
Inorden	1 2 3 5 30 18 42 7 25 10 12 15 20
Postorden	1 3 2 30 42 18 25 7 5 12 20 15 10
BFS	10 5 15 2 7 12 20 1 3 18 25 30 42

4.3. Contar Nodos totales

La función contarNodos() aplica el principio de división y conquista: si el nodo es nulo retorna 0; caso contrario retorna 1 (el nodo actual) más 1 conteo recursivo de ambos subárboles.

C++

```
int contarNodos(Nodo* raiz) {  
    if (raiz == nullptr) return 0;  
    return 1 + contarNodos(raiz->izquierda) + contarNodos(raiz->derecha);  
}
```

Java

```
public static int contarNodos(Nodo raiz) {  
    if (raiz == null) return 0;  
    return 1 + contarNodos(raiz.izquierda) + contarNodos(raiz.derecha);  
}
```

Tabla IV. Nodos Totales

Árbol	Nodos totales
Árbol original	7
Árbol modificado	11

4.4. Contar hojas

Una hoja es un nodo sin hijos. La función detecta y verifica que ambos punteros (izquierda y derecha) sean nulos. En este caso retorna 1; de lo contrario, propaga la búsqueda recursivamente.

C++

```
int contarHojas(Nodo* raiz) {  
    if (raiz == nullptr) return 0;  
    if (raiz->izquierda == nullptr && raiz->derecha == nullptr) return 1;  
    return contarHojas(raiz->izquierda) + contarHojas(raiz->derecha);  
}
```

Java

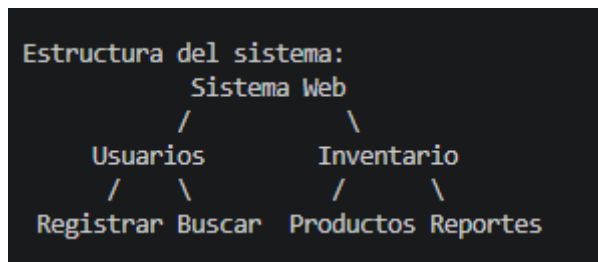
```
public static int contarHojas(Nodo raiz) {  
    if (raiz == null) return 0;  
    if (raiz.izquierda == null && raiz.derecha == null) return 1;  
    return contarHojas(raiz.izquierda) + contarHojas(raiz.derecha);  
}
```

Tabla V. Conteo de Hojas

Árbol	Hojas	Nodos totales
Árbol original	4	2, 7, 12, 20
Árbol modificado	6	1, 3, 7, 12, 18, 25

4.5. Aplicación al Proyecto Final

Se modela la arquitectura de un sistema web como un árbol binario. Cada nodo representa un módulo funcional del sistema y las relaciones padre-hijo reflejan la jerarquía de dependencias entre módulos.



Recorridos

4.5.1. Mostrar el menú principal- recorrido preorden

La función preorden presenta primero el módulo raíz y luego sus submódulos de manera jerárquica. Es el más intuitivo para construir un menú de navegación, donde el usuario ve primero el sistema completo y luego las opciones disponibles.

C++

```
void preordenWeb(ModuloWeb* raiz) {
    if (raiz == nullptr) return;
    cout << raiz->nombre << " -> ";
    preordenWeb(raiz->izquierda);
    preordenWeb(raiz->derecha);
}
```

Java

```
public static void preordenWeb(ModuloWeb raiz) {
    if (raiz == null) return;
    System.out.print(raiz.nombre + " -> ");
    preordenWeb(raiz.izquierda);
    preordenWeb(raiz.derecha);
}
```

4.5.2. Procesar primero los módulos internos- recorrido inorden

La función inorden es útil cuando los submódulos deben inicializarse o procesarse antes que el módulo padre. Por ejemplo, inicializar la base de datos de Productos y Reportes antes de levantar el módulo de inventario.

C++

```
void inordenWeb(ModuloWeb* raiz) {
    if (raiz == nullptr) return;
    inordenWeb(raiz->izquierda);
    cout << raiz->nombre << " -> ";
    inordenWeb(raiz->derecha);
}
```

Java

```
public static void inordenWeb(ModuloWeb raiz) {
    if (raiz == null) return;
    inordenWeb(raiz.izquierda);
    System.out.print(raiz.nombre + " -> ");
    inordenWeb(raiz.derecha);
}
```

4.5.3. Mostrar módulos nivel por nivel- BFS

La función BFS procesa los nodos por niveles usando una cola. Es ideal para mostrar la jerarquía completa del sistema de forma organizada, como en un organigrama o un panel de administración que agrupa los módulos por cada.

C++

```
void bfsWeb(ModuloWeb* raiz) {
    if (raiz == nullptr) return;
    queue<ModuloWeb*> cola;
    cola.push(raiz);
    int nivel = 1;
```

Java

```
public static void bfsWeb(ModuloWeb raiz) {
    if (raiz == null) return;
    Queue<ModuloWeb> cola = new LinkedList<>();
    cola.add(raiz);
    int nivel = 1;
```

4.5.4. Selección de Recorrido

Tabla VI. Selección de Recorrido del Sistema

Caso	Recorrido	Justificación
Menú principal de sistema	Preorden	Muestra primero la raíz antes de las opciones derivadas
Inicializador módulos por capas	Inorden	Procesa submódulos antes de activar el modulo padre
Panel de administración	BFS	Presenta la jerarquía nivel a nivel de forma clara

4.5.5. Comparación C++ y Java

Tabla VII. Comparación de Lenguajes

Aspecto	C++	Java
Tipo de puntero/referencia	Nodo*(puntero explícito)	Nodo(referencia implícita)
Valor nulo	nullptr	null
Estructura de cola	queue<Nodo*>de<queue>	Queue<Nodo> de java.util
Acceso a miembros	raiz->dato	raiz.dato
Salida en consola	cout<<raiz->dato	System.out.print(raiz,dato)
Gestión de memoria	Manual (new, delete)	Automática (Garbage Collector)
Resultado lógico	Idéntico	Idéntico

5. Conclusiones

Al implementar los recorridos preorden, inorden, postorden y BFS se permitió explorar los árboles binarios de formas distintas, cada uno con aplicaciones específicas como mostrar menús, ordenar datos o procesar niveles jerárquicos. Lo cual permitió que la recursiva de los recorridos hicieran el código más claro y natural.

Las funciones auxiliares nos demuestran que los árboles no solo almacenan datos, sino que también permiten extraer métricas útiles mediante recorridos recursivos. Lo cual es fundamental para sistemas que necesitan conocer el tamaño total o identificar elementos finales como módulos sin submódulos.

La aplicación al sistema web confirma que los árboles binarios modelan eficazmente la jerarquía de módulos, donde preorden muestra en menú principal, inorden procesa módulos internos primero y BFS despliega la estructura nivel por nivel. Esto valida la utilidad de los recorridos en problemas reales de organización de información.

6. Referencias

[1] Universidad Estatal de Milagro - UNEMI, "Estructura de Datos. Tema 1: Árboles y grafos," UNEMI ONLINE, Milagro, Ecuador, Informe Técnico FR0018/v1.0/18-11-2019, nov. 2019. [En línea]. Disponible: https://sga.unemi.edu.ec/media/recursotema/Documento_2020624153658.pdf

7. Anexos

Link GitHub: https://github.com/tituanadiana65-glitch/tarea2.2repo_recorridos_arboles_uta

Estructura del proyecto

```
tarea2.2 repo_recorridos_arboles_uta/
├── docs/
├── exercises/
├── moodle/
├── src/
│   ├── cpp/
│   │   ├── ejercicios.cpp
│   │   ├── main.cpp
│   │   ├── ejercicios.exe
│   │   └── recorridos.exe
│   └── java/
│       ├── Ejercicio.java
│       ├── Main.java
│       ├── ModuloWeb.class
│       ├── Nodo.class
│       └── ... (archivos .class generados)
├── LICENSE
├── README.md
└── README.md
```

Ejecución del código

----- Arbol original -----

Preorden: 10 5 2 7 15 12 20

Inorden: 2 5 7 10 12 15 20

Postorden: 2 7 5 12 20 15 10

BFS: 10 5 15 2 7 12 20

----- Arbol modificado -----

Nuevos nodos agregados: 1, 3, 18, 25, 30, 42

Preorden: 10 5 2 1 3 7 18 25 15 12 30 42 20

Inorden: 1 2 3 5 18 7 25 10 30 12 42 15 20

Postorden: 1 3 2 18 25 7 5 30 42 12 20 15 10

BFS: 10 5 15 2 7 12 20 1 3 18 25 30 42

----- Contar nodos totales -----

Total de nodos en el arbol: 13

----- Contar hojas -----

Total de hojas en el arbol: 7

----- Sistema Web -----

Estructura del sistema:

```
      Sistema Web
      /         \
    Usuarios     Inventario
    /  \         /  \
  Registrar Buscar Productos Reportes
```

1) Mostrar menu principal (Preorden):

Sistema Web -> Usuarios -> Registrar -> Buscar -> Inventario -> Productos -> Reportes -> NULL

2) Procesar primero modulos internos (Inorden):

Registrar -> Usuarios -> Buscar -> Sistema Web -> Productos -> Inventario -> Reportes -> NULL

3) Mostrar modulos nivel por nivel (BFS):

Nivel 1: Sistema Web

Nivel 2: Usuarios, Inventario

Nivel 3: Registrar, Buscar, Productos, Reportes